perplexity

0.10.3 ,

Model Name	Type	Size	Device	Tools	Cached
phi-4-mini	Chat	2.2 GB	GPU	●	●
deepseek-r1-1.5b	Chat	1.3 GB	GPU	○	○
deepseek-r1-14b	Chat	7.9 GB	GPU	○	○
deepseek-r1-7b	Chat	4.2 GB	GPU	○	○
gemma-4-e2b-it	Multimodal	5.8 GB	GPU	●	○
gpt-oss-20b	Chat	11.1 GB	GPU	○	○
ministral-3-3b-instruct-2512	Multimodal	3.6 GB	GPU	●	○
mistral-7b-v0.2	Chat	4.3 GB	GPU	○	○
mistral-nemo-12b-instruct	Chat	6.6 GB	GPU	●	○
nemotron-3.5-asr-streaming-0.6b	Speech	756 MB	GPU	○	○
nemotron-speech-streaming-en-0.6b	Speech	696 MB	GPU	○	○
nemotron-speech-streaming-es-0.6b	Speech	729 MB	GPU	○	○
olmo-3-7b-instruct	Chat	5.1 GB	GPU	●	○
parakeet-tdt-0.6b-v2	Speech	692 MB	GPU	○	○
phi-3-mini-128k	Chat	2.3 GB	GPU	○	○
phi-3-mini-4k	Chat	2.0 GB	GPU	○	○
phi-3.5-mini	Chat	1.9 GB	GPU	○	○
phi-4	Chat	8.8 GB	GPU	○	○
phi-4-mini-reasoning	Chat	2.5 GB	GPU	●	○
phi-4-reasoning	Chat	8.4 GB	GPU	○	○
qwen2.5-0.5b	Chat	366 MB	GPU	●	○
qwen2.5-1.5b	Chat	1019 MB	GPU	●	○
qwen2.5-14b	Chat	4.8 GB	GPU	●	○
qwen2.5-7b	Chat	4.8 GB	GPU	●	○
qwen2.5-coder-0.5b	Chat	365 MB	GPU	●	○
qwen2.5-coder-1.5b	Chat	1018 MB	GPU	●	○
qwen2.5-coder-14b	Chat	9.1 GB	GPU	●	○
qwen2.5-coder-7b	Chat	4.8 GB	GPU	●	○
qwen3-0.6b	Chat	492 MB	GPU	●	○
qwen3-1.7b	Chat	1.3 GB	GPU	●	○
qwen3-14b	Chat	9.1 GB	GPU	●	○
qwen3-4b	Chat	2.6 GB	GPU	●	○
qwen3-8b	Chat	5.5 GB	GPU	●	○
qwen3-embedding-0.6b	Embedding	478 MB	GPU	○	○
qwen3-embedding-8b	Embedding	5.5 GB	GPU	○	○

```
| qwen3-vl-2b-instruct | Multimodal | 2.1 GB | GPU | ● | ○ |
| qwen3-vl-4b-instruct | Multimodal | 3.5 GB | GPU | ● | ○ |
| qwen3-vl-8b-instruct | Multimodal | 6.0 GB | GPU | ● | ○ |
| qwen3.5-0.8b | Multimodal | 1.4 GB | GPU | ● | ○ |
| qwen3.5-2b | Multimodal | 2.9 GB | GPU | ● | ○ |
| qwen3.5-2b-text | Chat | 1.3 GB | GPU | ● | ○ |
| qwen3.5-4b | Multimodal | 4.1 GB | GPU | ● | ○ |
| qwen3.5-9b | Multimodal | 7.0 GB | GPU | ● | ○ |
| smollm3-3b | Chat | 2.0 GB | GPU | ● | ○ |
| whisper-base | Speech | 123 MB | GPU | ○ | ○ |
| whisper-large-v3-turbo | Speech | 803 MB | GPU | ○ | ○ |
| whisper-medium | Speech | 639 MB | GPU | ○ | ○ |
| whisper-small | Speech | 301 MB | GPU | ○ | ○ |
| whisper-tiny | Speech | 85 MB | GPU | ○ | ○ |
└──────────────────────────────────────────┴──────────────┴────────┴─────┴─────┘
```

Your setup is ready. For this proof of concept, use `qwen2.5-1.5b` as the LLM for test-case generation and `qwen3-vl-2b-instruct` as the VLM for screenshot review. Both are lightweight, GPU-listed, and marked with tool support; aliases let Foundry Local choose the best hardware variant automatically.[1] [2]

Recommended models

Purpose	Model alias	Why
Generate regression test cases	`qwen2.5-1.5b`	Small, fast, already supported, suitable for structured QA prompts
Analyze Playwright screenshots	`qwen3-vl-2b-instruct`	Multimodal vision model, only 2.1 GB, appropriate for UI screenshot checks
Optional stronger LLM later	`qwen3-4b`	Better reasoning while still relatively lightweight
Optional stronger VLM later	`qwen3-vl-4b-instruct`	Better visual understanding at 3.5 GB

Your catalog does **not** include `qwen3.5-vision`, so use `qwen3-vl-2b-instruct` instead. Qwen3-VL models support image input through chat-completion messages.[3] [4]

Update the client

In `foundry_client.py`, replace the model constants:

```
LLM_ALIAS = "qwen2.5-1.5b"
VLM_ALIAS = "qwen3-vl-2b-instruct"
```

Foundry Local accepts either an alias or a full model ID; using the alias is recommended because it selects the best variant for your GPU.[2] [5] [1]

Download and verify models

Run:

```
foundry model download qwen2.5-1.5b
foundry model download qwen3-vl-2b-instruct
```

Then verify each model can load:

```
foundry model load qwen2.5-1.5b
foundry model load qwen3-vl-2b-instruct
foundry model list
```

You can also test the LLM directly:

```
foundry chat qwen2.5-1.5b
```

Type a prompt, confirm it responds, then exit. The CLI supports `foundry chat <alias>` for interactive local chat sessions. [6] [1]

Start the local API server

For the Python/OpenAI-compatible proof of concept, start the server:

```
foundry server start
```

Check its status and endpoint:

```
foundry status
```

In Foundry Local 0.10.3, the newer command is `foundry server start`; older `foundry service start` syntax was replaced. [7] [8]

Simplified Python client

Because you are using Foundry Local 0.10.3 with its server, use this simpler `foundry_client.py`. It avoids SDK version differences by calling the local OpenAI-compatible endpoint directly:

```
import base64
import json
import os
import re
from pathlib import Path

import httpx

LLM_ALIAS = "qwen2.5-1.5b"
```

```
VLM_ALIAS = "qwen3-vl-2b-instruct"
BASE_URL = os.getenv("FOUNDRY_LOCAL_BASE_URL", "http://localhost:5272/v1")


def chat(messages, model_alias, temperature=0.2):
    response = httpx.post(
        f"{BASE_URL}/chat/completions",
        json={
            "model": model_alias,
            "messages": messages,
            "temperature": temperature,
        },
        timeout=180,
    )
    response.raise_for_status()
    content = response.json()["choices"][^0]["message"]["content"]
    return extract_json(content)


def extract_json(text):
    match = re.search(r"\{.*\}", text, re.DOTALL)
    if not match:
        raise ValueError(f"Model did not return JSON:\n{text}")
    return json.loads(match.group())


def generate_test_cases(feature_description):
    prompt = f"""
Generate regression test cases for this feature:

{feature_description}

Return only valid JSON:
{{
  "test_cases": [
    {{
      "title": "",
      "priority": "high|medium|low",
      "steps": [],
      "test_data": {{}},
      "expected_result": ""
    }}
  ]
}}
"""

    return chat(
        [
            {"role": "system", "content": "You are a precise QA test designer."},
            {"role": "user", "content": prompt},
        ],
        LLM_ALIAS,
    )


def analyze_screenshot(screenshot_path, expected_rules):
```

```
    image_b64 = base64.b64encode(Path(screenshot_path).read_bytes()).decode()

    prompt = f"""
Review this web-page screenshot for regression testing.

Expected rules:
{expected_rules}

Return only valid JSON:
{{
  "status": "pass|fail",
  "issues": [],
  "summary": ""
}}

Only report visible, factual UI issues. Do not invent issues.
"""

    return chat(
        [
            {
                "role": "user",
                "content": [
                    {"type": "text", "text": prompt},
                    {
                        "type": "image_url",
                        "image_url": {
                            "url": f"{image_b64}"
                        },
                    },
                ],
            }
        ],
        VLM_ALIAS,
        temperature=0.0,
    )
```

If `foundry status` shows a different port, set it before running pytest:

```
$env:FOUNDRY_LOCAL_BASE_URL = "http://localhost:YOUR_PORT/v1"
```

Update the test file

Replace the imports and fixture in `tests/test_ai_regression.py` with:

```
import json
from pathlib import Path

import pytest
from playwright.sync_api import Page

from foundry_client import analyze_screenshot, generate_test_cases
```

```
SCREENSHOTS = Path("screenshots")
SCREENSHOTS.mkdir(exist_ok=True)
TARGET_URL = "https://playwright.dev/"


def test_llm_generates_login_regression_cases():
    result = generate_test_cases(
        "A login page with email, password, sign-in button, error messages, "
        "and forgot-password link."
    )

    assert "test_cases" in result
    assert len(result["test_cases"]) > 0

    Path("reports").mkdir(exist_ok=True)
    Path("reports/llm_test_cases.json").write_text(
        json.dumps(result, indent=2), encoding="utf-8"
    )


def test_vlm_checks_playwright_homepage(page: Page):
    page.goto(TARGET_URL)
    page.screenshot(path=str(SCREENSHOTS / "home.png"), full_page=True)

    result = analyze_screenshot(
        SCREENSHOTS / "home.png",
        """
        - The page must load with visible content.
        - The page must not show a browser error page.
        - The page must contain navigation or documentation links.
        """,
    )

    Path("reports").mkdir(exist_ok=True)
    Path("reports/vlm_home_review.json").write_text(
        json.dumps(result, indent=2), encoding="utf-8"
    )

    print("\nVLM result:", json.dumps(result, indent=2))

    assert result.get("status") in ("pass", "fail")
    if result.get("status") == "fail":
        assert result.get("issues"), "VLM reported failure without listing issues."
```

Run it

In one PowerShell window:

```
foundry server start
foundry status
```

In a second PowerShell window:

```
cd ai-regression-tests
.\.venv\Scripts\Activate.ps1
pytest -s -v
```

Expected artifacts:

- `reports/llm_test_cases.json` — regression test cases generated by `qwen2.5-1.5b`
- `screenshots/home.png` — Playwright screenshot of the public demo site
- `reports/vlm_home_review.json` — screenshot analysis from `qwen3-vl-2b-instruct` [1] [3]

If something fails

Error	Likely fix
`Model not found`	Use the exact alias from `foundry model list`; your VLM alias is `qwen3-vl-2b-instruct`, not `qwen3.5-vision`
Connection refused	Run `foundry server start`, then copy the URL shown by `foundry status` into `FOUNDRY_LOCAL_BASE_URL`
VLM returns text instead of JSON	Keep the strict prompt and `extract_json` function; if needed, increase `max_tokens`
Playwright browser missing	Run `playwright install` inside the activated virtual environment
Test is slow on first run	The first model load/download can take time; later runs should be faster

Run the commands above and paste any error output or the contents of `reports/vlm_home_review.json`; I'll help you interpret the VLM result and tune the prompts. [2] [7] [1]

⁂

1. https://learn.microsoft.com/en-us/azure/foundry-local/reference/reference-cli
2. https://learn.microsoft.com/en-us/azure/ai-foundry/foundry-local/reference/reference-cli
3. https://ai.azure.com/catalog/models/qwen-qwen3-vl-2b-thinking
4. https://huggingface.co/Qwen/Qwen3-VL-2B-Instruct
5. https://learn.microsoft.com/en-ie/azure/foundry-local/reference/reference-sdk-legacy
6. https://learn.microsoft.com/fi-fi/azure/foundry-local/how-to/how-to-use-foundry-local-cli
7. https://github.com/microsoft/foundry-local/releases/tag/cli-preview-0.10.3
8. https://github.com/microsoft/foundry-local/releases
9. https://learn.microsoft.com/en-nz/azure/foundry-local/how-to/how-to-use-tool-calling-with-foundry-local
10. https://learn.microsoft.com/en-au/azure/foundry-local/get-started
11. https://github.com/retkowsky/foundry-local
12. https://github.com/mhackermsft/wslcontainerdesktop/blob/main/docs/FOUNDRY-LOCAL.md
13. https://github.com/microsoft/Foundry-Local/releases

14. https://github.com/microsoft-foundry/Foundry-Local-Lab/blob/main/labs/part1-getting-started.md

15. https://learn.microsoft.com/en-us/windows/ai/foundry-local/get-started